

FRAUD DETECTION SQL ANALYSIS

Project Documentation — Source Edition

Built from verified SQL files | Nigerian Banking & Fintech Scenario | 2024 Dataset

Analyst: Akinwale David (SHADXW) | August 2026

Tool: MySQL | Files: fraud_analysis.sql, questions 1–4 SQL files

6,270

Transactions

120

Accounts

15

Merchants

1.37%

Fraud Rate

4

SQL Files

Table of Contents

1.	Data Cleaning Process — fraud_analysis.sql	3
2.	Q1: Fraud Rate by Account Segment, Risk Profile & Channel	5
3.	Q2: Fraud Pattern Breakdown	8
4.	Q3: High-Risk / Repeat Offender Accounts	10
5.	Q4: Merchant & Channel Exposure	12
6.	Recommendations	14

Data Cleaning Process

Source file: *fraud_analysis.sql*

The cleaning script addresses four concerns in the following order: duplicate detection, data standardisation (whitespace trimming), date-column conversion from text to DATETIME, and NULL value auditing. All operations run directly on the source tables.

Step 1 — Check for Duplicates

A subquery wraps the full transactions table and applies ROW_NUMBER() as a window function, partitioning across every column in the table. Any row whose assigned row number exceeds 1 is a true duplicate of a prior record.

```
SELECT *
FROM (
  SELECT *,
  ROW_NUMBER() OVER(PARTITION BY transaction_id, account_id, transaction_date,
  amount_ngn, merchant_id, merchant_name, merchant_category, transaction_channel,
  transaction_location, account_home_city, is_fraud, fraud_flag_reason) AS r_num
  FROM transactions
) AS Duplicates
WHERE r_num > 1;
```

ROW_NUMBER() partitions on all twelve columns simultaneously, so only rows identical in every field receive a number greater than 1. Selecting * on the outer query returns the full duplicate record for inspection rather than just a count, giving the analyst visibility into which rows are affected before any deletion decision.

Step 2 — Standardise the Data (Trim Whitespace)

TRIM() is applied to every text column across all three tables. The script runs three separate UPDATE statements on accounts, then merchants, then transactions.

accounts table

```
UPDATE accounts
SET
  account_id = TRIM(account_id),
  city = TRIM(city),
  account_type = TRIM(account_type),
  risk_profile = TRIM(risk_profile);
```

merchants table

```
UPDATE merchants
SET
  merchant_id = TRIM(merchant_id),
  merchant_name = TRIM(merchant_name),
  merchant_category = TRIM(merchant_category),
  country = TRIM(country);
```

transactions table

```
UPDATE transactions
SET
transaction_id = TRIM(transaction_id),
account_id = TRIM(account_id),
merchant_id = TRIM(merchant_id),
merchant_name = TRIM(merchant_name),
merchant_category = TRIM(merchant_category),
transaction_channel = TRIM(transaction_channel),
transaction_location = TRIM(transaction_location),
account_home_city = TRIM(account_home_city),
fraud_flag_reason = TRIM(fraud_flag_reason);
```

Leading and trailing spaces cause silent mismatches in JOIN and GROUP BY operations: two values that look identical on screen fail to match when one carries an invisible space character. TRIM() resolves this across all tables before any analysis runs.

Step 3 — Convert the Date Column from Text to DATETIME

The transaction_date column was stored as a plain text string. The script converts it to a proper DATETIME type through four sub-steps: inspect, add a new column, populate it, verify parse success, then swap out the original.

3a — Inspect date values for format inconsistencies

```
SELECT transaction_date, COUNT(*) AS occurrences
FROM transactions
GROUP BY transaction_date
HAVING COUNT(*) = 1
ORDER BY transaction_date;
```

HAVING COUNT(*) = 1 surfaces dates that appear only once, helping spot formatting outliers that would cause STR_TO_DATE to silently return NULL.

3b — Add and populate the new DATETIME column

```
ALTER TABLE transactions ADD COLUMN txn_date DATETIME;

UPDATE transactions
SET txn_date = STR_TO_DATE(transaction_date, '%Y-%m-%d %H:%i:%s');
```

3c — Check for parse failures

```
SELECT transaction_id, txn_date
FROM transactions
WHERE txn_date IS NULL;
```

STR_TO_DATE returns NULL when an input string does not match the format mask. This check catches those rows before the original column is dropped.

3d — Swap columns

```
ALTER TABLE transactions DROP COLUMN transaction_date;
ALTER TABLE transactions RENAME COLUMN txn_date TO transaction_date;
```

Dropping the original string column and renaming the typed replacement preserves the original column name, keeping downstream queries and scripts compatible.

Step 4 — Audit NULL Values Across All Tables

Three SELECT queries run a full NULL count on every column in each table. Each column gets its own CASE WHEN expression so the audit result arrives as a single summary row, covering all columns in one pass.

accounts table

```
SELECT
SUM(CASE WHEN account_id IS NULL THEN 1 ELSE 0 END) AS null_id,
SUM(CASE WHEN age IS NULL THEN 1 ELSE 0 END) AS null_age,
SUM(CASE WHEN city IS NULL THEN 1 ELSE 0 END) AS null_city,
SUM(CASE WHEN account_type IS NULL THEN 1 ELSE 0 END) AS null_acc_type,
SUM(CASE WHEN avg_monthly_balance IS NULL THEN 1 ELSE 0 END) AS null_bal,
SUM(CASE WHEN risk_profile IS NULL THEN 1 ELSE 0 END) AS null_risk_profile
FROM accounts;
```

merchants table

```
SELECT
SUM(CASE WHEN merchant_id IS NULL THEN 1 ELSE 0 END) AS null_id,
SUM(CASE WHEN merchant_name IS NULL THEN 1 ELSE 0 END) AS null_name,
SUM(CASE WHEN merchant_category IS NULL THEN 1 ELSE 0 END) AS null_category,
SUM(CASE WHEN is_registered IS NULL THEN 1 ELSE 0 END) AS null_register,
SUM(CASE WHEN country IS NULL THEN 1 ELSE 0 END) AS null_country
FROM merchants;
```

transactions table

```
SELECT
SUM(CASE WHEN transaction_id IS NULL THEN 1 ELSE 0 END) AS null_id,
SUM(CASE WHEN account_id IS NULL THEN 1 ELSE 0 END) AS null_acc_id,
SUM(CASE WHEN merchant_id IS NULL THEN 1 ELSE 0 END) AS null_merch_id,
SUM(CASE WHEN merchant_name IS NULL THEN 1 ELSE 0 END) AS null_name,
SUM(CASE WHEN merchant_category IS NULL THEN 1 ELSE 0 END) AS null_category,
SUM(CASE WHEN transaction_channel IS NULL THEN 1 ELSE 0 END) AS null_channel,
SUM(CASE WHEN transaction_location IS NULL THEN 1 ELSE 0 END) AS null_location,
SUM(CASE WHEN account_home_city IS NULL THEN 1 ELSE 0 END) AS null_home,
SUM(CASE WHEN is_fraud IS NULL THEN 1 ELSE 0 END) AS null_fraud,
SUM(CASE WHEN fraud_flag_reason IS NULL THEN 1 ELSE 0 END) AS null_reason,
SUM(CASE WHEN transaction_date IS NULL THEN 1 ELSE 0 END) AS null_date
FROM transactions;
```

SUM(CASE WHEN ... IS NULL THEN 1 ELSE 0 END) converts each NULL check into a binary integer per row, then sums across all rows. The output is a count of missing values per column delivered in one result row, faster to read than running one query per column.

Q1

Fraud Rate by Account Segment, Risk Profile & Channel

Source file: `fraud_analysis_question1.sql`

The Problem

The fraud team needs to know where fraud concentrates within the customer base. Three dimensions matter: the account product type (Savings, Current, Salary), the bank's own risk classification of the customer (Low, Medium, High), and the channel through which transactions are made (Mobile App, Web, POS, ATM, USSD). A second question sits alongside the rate question: what share of the bank's total transaction value is being lost to fraud?

Process

The script opens with a referential integrity check, joining accounts and transactions and filtering for IDs that exist in one table but not the other. Zero rows returned confirms the JOIN is safe. Three separate GROUP BY queries then compute the fraud rate segmented by account_type, by risk_profile, and by transaction_channel. A final block calculates overall total transaction value, total fraud value, and the percentage that fraud represents, then repeats those calculations segmented by account_type and channel.

Referential integrity check

```
SELECT
a.account_id AS accts_id,
t.account_id AS txns_id
FROM accounts a
JOIN transactions t ON a.account_id = t.account_id
WHERE t.account_id NOT IN (SELECT a.account_id FROM accounts)
OR a.account_id NOT IN (SELECT t.account_id FROM transactions);
```

Fraud rate by account type

```
SELECT account_type,
ROUND(100 * SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS fraud_rate
FROM accounts a
JOIN transactions t ON a.account_id = t.account_id
GROUP BY account_type
ORDER BY fraud_rate DESC;
```

Fraud rate by risk profile

```
SELECT risk_profile,
ROUND(100 * SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS fraud_rate
FROM accounts a
JOIN transactions t ON a.account_id = t.account_id
GROUP BY risk_profile
ORDER BY fraud_rate DESC;
```

Fraud rate by transaction channel

```
SELECT transaction_channel,
ROUND(100 * SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS fraud_rate
FROM accounts a
JOIN transactions t ON a.account_id = t.account_id
GROUP BY transaction_channel
```

```
ORDER BY fraud_rate DESC;
```

Total and fraud transaction values (overall)

```
-- Total amount transacted
SELECT ROUND(SUM(amount_ngn), 2) AS Total_txns_val
FROM transactions;

-- Total value of fraud transactions only
SELECT ROUND(SUM(amount_ngn), 2) AS Fraud_txns_val
FROM transactions
WHERE is_fraud = 1;

-- Fraud value as a percentage of all transaction value
SELECT
ROUND((100 *
(SELECT ROUND(SUM(amount_ngn), 2) FROM transactions WHERE is_fraud = 1))
/ ROUND(SUM(amount_ngn), 2), 2) AS Fraud_txns_val_pct
FROM transactions;
```

Fraud value share segmented by account type and channel

```
-- By account type
SELECT account_type,
ROUND((100 *
(SELECT ROUND(SUM(amount_ngn), 2) FROM transactions WHERE is_fraud = 1))
/ ROUND(SUM(amount_ngn), 2), 2) AS Fraud_txns_val_pct
FROM accounts a
JOIN transactions t ON a.account_id = t.account_id
GROUP BY account_type;

-- By channel
SELECT transaction_channel,
CONCAT(ROUND((100 *
(SELECT ROUND(SUM(amount_ngn), 2) FROM transactions WHERE is_fraud = 1))
/ ROUND(SUM(amount_ngn), 2), 2), '%') AS Fraud_txns_val_pct
FROM accounts a
JOIN transactions t ON a.account_id = t.account_id
GROUP BY transaction_channel
ORDER BY Fraud_txns_val_pct DESC;
```

Metrics Used

Metric	Formula	Why it was used
Fraud rate	$100 * \text{SUM}(\text{CASE WHEN is_fraud}=1 \text{ THEN } 1 \text{ ELSE } 0 \text{ END}) / \text{COUNT}(*)$	Normalises fraud occurrence to segment size, making Savings, Current, and Salary directly comparable regardless of how many customers each type has.
Total transaction value	$\text{ROUND}(\text{SUM}(\text{amount_ngn}), 2)$	Establishes the full monetary base of each segment or channel. Provides the denominator for the fraud value share percentage.

Fraud transaction value	SUM(amount_ngn) WHERE is_fraud = 1 (correlated subquery)	Isolates monetary value flowing through confirmed fraud cases only, calculated separately from the main aggregate via a correlated subquery.
Fraud value share (%)	100 * fraud_value / total_value	Converts transaction volume into monetary exposure per segment. A segment with a moderate fraud rate but high-value transactions loses more than a high-rate, low-value one — this metric surfaces that difference.

Insights

What the data shows

- Current accounts carry the highest fraud rate among account types. Higher transfer limits make them more attractive targets for large fraudulent transactions.
- High risk profile accounts record the highest per-segment fraud rate, confirming the bank's classification carries some predictive power.
- Medium risk accounts generate more total fraud cases than high risk accounts because the medium-risk customer base is significantly larger.
- Web and Mobile App post the highest fraud rates among all transaction channels, confirming digital card-not-present transactions as the primary attack surface.
- Fraud's share of total transaction value exceeds its share of transaction count, meaning fraudulent transactions are on average larger than legitimate ones.

In plain language

Current account holders lose more money to fraud than savings or salary customers because fraudsters target accounts where transfers can be larger. The bank's risk labels work to a point: high-risk customers do get hit more often. But medium-risk customers cause the most damage in total, simply because there are far more of them. Online banking and mobile apps are where most fraud occurs. The fraud value share number is the most important: each fraudulent transaction takes more money than a typical legitimate one, so the financial damage is disproportionate to the case count.

Source file: `fraud_analysis_question2.sql`

The Problem

Fraud is not uniform. Each attack method has a different frequency, a different typical transaction size, and a different total monetary impact. The fraud team needs to rank the patterns to decide which detection rules to tighten first and where to concentrate investigation capacity.

Process

A single query filters transactions to `is_fraud = 1`, then groups by `fraud_flag_reason`. All aggregations run in one pass: count, percentage share via a window function, total value, average amount, maximum, and minimum. Results are ordered by `fraud_count` descending to rank patterns by frequency.

```
SELECT
  fraud_flag_reason,
  COUNT(*) AS Fraud_count,
  CONCAT(ROUND(COUNT(*) / SUM(COUNT(*) OVER()) * 100, 2), '%') AS pct_of_all_fraud,
  ROUND(SUM(amount_ngn), 2) AS total_transacted,
  ROUND(AVG(amount_ngn), 2) AS avg_amt_per_txn,
  ROUND(MAX(amount_ngn), 2) AS maximum_amt_per_fraud_type,
  ROUND(MIN(amount_ngn), 2) AS minimum_amt_per_fraud_type
FROM transactions
WHERE is_fraud = 1
GROUP BY fraud_flag_reason
ORDER BY Fraud_count DESC;
```

Metrics Used

Metric	Formula	Why it was used
Fraud count	<code>COUNT(*)</code>	Raw frequency per fraud pattern. Primary ranking dimension — shows which attack method appears most often in the flagged population.
% of all fraud	<code>COUNT(*) / SUM(COUNT(*) OVER()) * 100</code>	Window function computes the grand total in the same GROUP BY pass, giving each pattern's share of all fraud without a subquery or second table scan.
Total transacted	<code>SUM(amount_ngn)</code>	Total monetary exposure per pattern. High-value, low-count patterns are invisible on a frequency-only view; this column surfaces them.
Avg amount per txn	<code>AVG(amount_ngn)</code>	Typical transaction size per fraud type. A high average signals larger single-incident losses and guides where to set monetary alert thresholds.

Max / Min amount	MAX(amount_ngn) MIN(amount_ngn)	Reveals range and outliers within each pattern. A large gap between max and min signals one fraud type covers a wide spread of transaction sizes.
-------------------------	---------------------------------	---

Insights

What the data shows

- Geographic anomaly is the most frequent fraud pattern, appearing in the largest share of all flagged transactions, showing location-based attacks are the dominant method.
- Round-amount structuring carries the highest average and maximum transaction values, making it the most damaging pattern per incident even if it ranks lower by count.
- Velocity spike transactions are frequent but carry the lowest average amounts, consistent with fraudsters keeping individual transactions small to avoid auto-blocking.
- Odd-hour large transactions combine a moderate count with high individual amounts, producing significant total exposure among the top patterns by monetary value.
- The CONCAT('%') wrapping on the percentage column makes the output human-readable in a direct export without requiring a BI tool to format it.

In plain language

Six fraud methods were detected. The most common is when a card is used in a city or country the account holder has never visited. The costliest type involves moving large, round amounts in single transfers. A separate group of attacks runs overnight between 1am and 4am when account holders are unlikely to notice an alert. Velocity fraud uses many small rapid transactions to drain money in pieces rather than one large hit. The frequency ranking tells the team where most attacks originate; the average amount column tells them which attacks inflict the most damage per incident.

Source file: `fraud_analysis_question3.sql`

The Problem

A single fraud event on an account could be a one-time compromise. Two or more events on the same account signal either an unresolved breach or a deliberate bad actor. The fraud team needs a watchlist of repeat-offender accounts, the channels they exploit, and a comparison between their fraud transaction size and their normal spending to distinguish account takeover from owner misconduct.

Process

The query uses four CTEs chained together. `fraud_count` groups all fraud transactions by account and counts them. `repeat_offenders` filters to accounts with more than one fraud event. `avg_legit_txn_amt` joins back to transactions, restricts to `is_fraud = 0`, and calculates the average legitimate transaction amount per account. `channels_used` uses `GROUP_CONCAT` with `DISTINCT` to collapse each account's fraud channel list into one readable cell. The final `SELECT` joins all four CTEs and computes `avg_fraud_txn_amt` and `Fraud_to_legit_ratio` as correlated subqueries.

```
WITH fraud_count AS (
  SELECT account_id, COUNT(*) AS Total_fraud_txns
  FROM transactions WHERE is_fraud = 1
  GROUP BY account_id
),
repeat_offenders AS (
  SELECT account_id, Total_fraud_txns
  FROM fraud_count WHERE Total_fraud_txns > 1
),
avg_legit_txn_amt AS (
  SELECT t.account_id,
  ROUND(AVG(t.amount_ngn), 2) AS avg_legit_amt
  FROM transactions t
  JOIN repeat_offenders ro ON t.account_id = ro.account_id
  WHERE is_fraud = 0
  GROUP BY account_id
),
channels_used AS (
  SELECT t.account_id,
  GROUP_CONCAT(DISTINCT t.transaction_channel
  ORDER BY t.transaction_channel SEPARATOR ', ') AS channels
  FROM transactions t
  JOIN repeat_offenders ro ON t.account_id = ro.account_id
  WHERE is_fraud = 1
  GROUP BY account_id
)
SELECT ro.account_id, a.account_type, a.risk_profile,
ro.Total_fraud_txns, cu.channels, al.avg_legit_amt,
ROUND((SELECT AVG(amount_ngn) FROM transactions t
WHERE t.account_id = ro.account_id AND is_fraud = 1), 2) AS avg_fraud_txn_amt,
ROUND((SELECT AVG(amount_ngn) FROM transactions t
```

```

WHERE t.account_id = ro.account_id AND is_fraud = 1)
/ NULLIF(al.avg_legit_amt, 0), 2) AS Fraud_to_legit_ratio
FROM repeat_offenders ro
JOIN accounts a ON ro.account_id = a.account_id
JOIN avg_legit_txn_amt al ON ro.account_id = al.account_id
JOIN channels_used cu ON ro.account_id = cu.account_id
ORDER BY Total_fraud_txns;

```

Metrics Used

Metric	Formula	Why it was used
Total fraud transactions	COUNT(*) WHERE is_fraud = 1 GROUP BY account_id	Counts fraud events per account. The filter Total_fraud_txns > 1 in the repeat_offenders CTE defines the watchlist threshold.
Avg legitimate amount	AVG(amount_ngn) WHERE is_fraud = 0 per repeat-offender account	Establishes the normal spending baseline per account. Serves as the denominator in the ratio and the comparison reference against fraud transaction sizes.
Avg fraud amount	Correlated subquery: AVG(amount_ngn) WHERE is_fraud = 1	Captures the typical size of fraudulent transactions on the same account, enabling direct comparison against the legitimate baseline.
Fraud-to-legit ratio	avg_fraud_amt / NULLIF(avg_legit_amt, 0)	A ratio above 1.0 means fraud transactions are larger than normal spending — the primary indicator of external account takeover. NULLIF prevents division-by-zero where an account has no legitimate transaction history.
Channels used	GROUP_CONCAT(DISTINCT channel ORDER BY channel SEPARATOR ', ')	Collapses the full channel list into one readable cell per account. DISTINCT removes repetitions when the same channel was used multiple times.

Insights

What the data shows

- Several accounts appear in the fraud table more than once, confirming that certain accounts are either actively compromised or belong to deliberate bad actors.
- The Fraud_to_legit_ratio exceeds 1.0 for most repeat offenders, meaning fraudulent transactions are consistently larger than the account holder's normal spending baseline. This is the strongest indicator of external account takeover.
- Web and Mobile App dominate the channels column for repeat offenders, while POS and ATM appearances are minimal, placing digital channels as the primary breach surface.
- Several repeat offenders carry Low or Medium risk profile labels in the accounts table, showing the bank's risk scoring failed to identify these accounts as vulnerable before fraud events accumulated.

In plain language

Certain accounts have been hit by fraud more than once. When the same account appears repeatedly in the fraud records it almost always means the login credentials were stolen and the attacker kept returning. The comparison between fraud amounts and normal spending confirms this: fraudulent transfers are bigger than anything the real account owner normally sends. These accounts need to be frozen immediately, passwords reset, and customers contacted to verify their identity. The fact that several of these accounts carried a low-risk label is a serious gap: repeated fraud occurred below the radar of the existing monitoring system.

Source file: `fraud_analysis_question4.sql`

The Problem

Fraud does not spread evenly across all merchants and payment channels. Concentrating monitoring and friction controls on the highest-risk merchants and channels produces the most impact with the least disruption to legitimate customers. The question asks which merchants and channels carry the most fraud by count and by value, and what proportion of each merchant's total volume is fraudulent.

Process

The script runs two separate query blocks. The first uses two CTEs to calculate total channel amounts and fraud channel amounts independently, then joins them to derive a fraud percentage per channel. The second query joins transactions to merchants and accounts, restricts to `is_fraud = 1` in the WHERE clause, and uses correlated subqueries inside SELECT to calculate total amount and fraud amount per merchant from the full transaction table. GROUP_CONCAT with DISTINCT captures every account and channel associated with fraud at each merchant.

Channel exposure query

```
WITH total_amt_per_channel AS (  
  SELECT transaction_channel,  
         ROUND(SUM(amount_ngn), 2) AS Total_amt  
  FROM transactions  
  GROUP BY transaction_channel  
) ,  
fraud_amt_per_channel AS (  
  SELECT transaction_channel,  
         ROUND(SUM(amount_ngn), 2) AS Fraud_amt  
  FROM transactions  
  WHERE is_fraud = 1  
  GROUP BY transaction_channel  
)  
SELECT  
  tc.transaction_channel,  
  tc.Total_amt,  
  fc.Fraud_amt,  
  ROUND(100 * fc.Fraud_amt / tc.Total_amt, 2) AS Fraud_pct  
FROM total_amt_per_channel tc  
JOIN fraud_amt_per_channel fc  
ON tc.transaction_channel = fc.transaction_channel;
```

Merchant exposure query

```
SELECT  
  m.merchant_name,  
  m.merchant_id,  
  m.is_registered,  
  GROUP_CONCAT(DISTINCT t.account_id  
    ORDER BY t.account_id SEPARATOR ', ' ) AS merchant_accounts,  
  GROUP_CONCAT(DISTINCT t.transaction_channel
```

```

ORDER BY t.transaction_channel SEPARATOR ', ' ) AS Channels_Used,
COUNT(DISTINCT t.account_id) AS No_of_accts,
ROUND((SELECT SUM(t.amount_ngn) FROM transactions t
WHERE t.merchant_id = m.merchant_id), 2) AS Total_amt,
ROUND((SELECT SUM(t.amount_ngn) FROM transactions t
WHERE t.merchant_id = m.merchant_id
AND is_fraud = 1), 2) AS Fraud_amt,
ROUND((100 *
(SELECT SUM(t.amount_ngn) FROM transactions t
WHERE t.merchant_id = m.merchant_id AND is_fraud = 1) /
(SELECT SUM(t.amount_ngn) FROM transactions t
WHERE t.merchant_id = m.merchant_id)), 2) AS Pct_Fraud_comp
FROM transactions t
JOIN merchants m ON t.merchant_id = m.merchant_id
JOIN accounts a ON t.account_id = a.account_id
WHERE t.is_fraud = 1
GROUP BY m.merchant_name, m.merchant_id, m.is_registered
ORDER BY No_of_accts DESC;

```

Metrics Used

Metric	Formula	Why it was used
Channel fraud %	100 * Fraud_amt / Total_amt per channel	Measures the share of each channel's total money flow that is fraudulent. Lets the team compare channels of different volumes on a level basis.
Merchant accounts	GROUP_CONCAT(DISTINCT account_id ORDER BY account_id SEPARATOR ', ')	Lists every account that transacted fraudulently at this merchant. Reveals whether fraud comes from one bad actor or spans multiple customers.
No. of accounts	COUNT(DISTINCT account_id)	A higher count means fraud at this merchant is broad across the customer base. Used as the primary ORDER BY key to surface the widest-impact merchants first.
Total amount	Correlated subquery: SUM(amount_ngn) for all txns at merchant	Provides the full monetary base for each merchant so fraud composition can be expressed as a percentage of total revenue flow through that merchant.
Fraud composition %	100 * Fraud_amt / Total_amt per merchant	Primary merchant risk signal. A value near 100% means the merchant processes almost no legitimate transactions and should be blocked outright.

Insights

What the data shows

- Unregistered merchants (is_registered = 0) record fraud compositions approaching or reaching 100%, meaning essentially every transaction through them is fraudulent.
- Crypto exchanges and wire transfer merchants carry the highest Fraud_amt values despite lower transaction counts, making them the largest monetary risk per merchant.
- The Web channel posts the highest Fraud_pct among all channels, confirming it as the primary digital attack surface across the entire transaction portfolio.
- The GROUP_CONCAT account list shows that fraud at high-risk merchants spans multiple distinct customers, ruling out single-account compromise and pointing to merchant-level or network-level fraud schemes.
- High-volume everyday merchants such as grocery stores and utility providers carry near-zero fraud composition, confirming that fraud concentrates in a narrow subset of the merchant base rather than spreading across the portfolio.

In plain language

Fraud does not happen everywhere. It clusters around a small number of specific payment destinations. Merchants the bank has not registered are almost entirely fraudulent: nearly every naira sent to them is stolen money. Crypto and international wire transfer merchants take the largest amounts. Online banking is the riskiest payment channel across the board. Everyday merchants such as supermarkets and electricity providers are clean. The bank can stop the majority of fraud by blocking or adding verification to a handful of merchant categories and the web channel, without affecting the experience of the typical customer paying a bill or buying groceries.

Recommendations

Three recommendations follow, ordered from minimum viable to highest impact. Each is grounded directly in a finding from the four analytical questions.

FAIR

Add OTP confirmation for round-amount transfers above 200,000 NGN

Q2 shows round-amount structuring carries the highest average and maximum transaction values among all fraud patterns. Requiring a one-time password for any transfer where the amount is an exact multiple of 100,000 NGN adds one friction step that the vast majority of legitimate customers will never encounter. It targets the highest-value fraud type directly and costs little to implement through the existing authentication layer. The limitation: it addresses only one pattern and does nothing about geographic anomalies or odd-hour attacks, leaving most fraud volume untouched.

GOOD

Block unregistered merchants and activate geo-velocity rules

Q4 shows unregistered merchants (`is_registered = 0`) process fraud at rates approaching 100%. The bank already holds this data and can block these destinations outright with zero false-positive risk to legitimate customers. Alongside this, Q2 ranks geographic anomaly as the single most common fraud pattern. A geo-velocity rule flags any transaction where the current location is physically impossible to reach from the previous location within the time gap between them. Both controls are rule-based, require no machine learning, and deploy through the existing transaction monitoring system. Together they address the two highest-frequency fraud patterns identified in the analysis.

BEST

Freeze repeat-offender accounts, force re-KYC, and recalibrate risk scoring

Q3 produces a concrete watchlist of accounts with more than one fraud event and `Fraud_to_legit_ratios` above 1.0. These accounts need outbound transfers suspended immediately. Affected customers should be contacted directly, passwords reset, and re-KYC completed before access is restored. Beyond the immediate account actions, Q3 reveals that several repeat-offender accounts carried Low or Medium risk labels when the fraud occurred. The scoring model failed to predict their vulnerability. The fraud team should feed the characteristics of these accounts back into the scoring criteria: transaction velocity, channel mix, geographic spread, and average amounts relative to balance. This recommendation addresses the root cause. The first two recommendations block specific fraud routes; this one closes the monitoring gap that allowed repeat fraud to accumulate undetected.